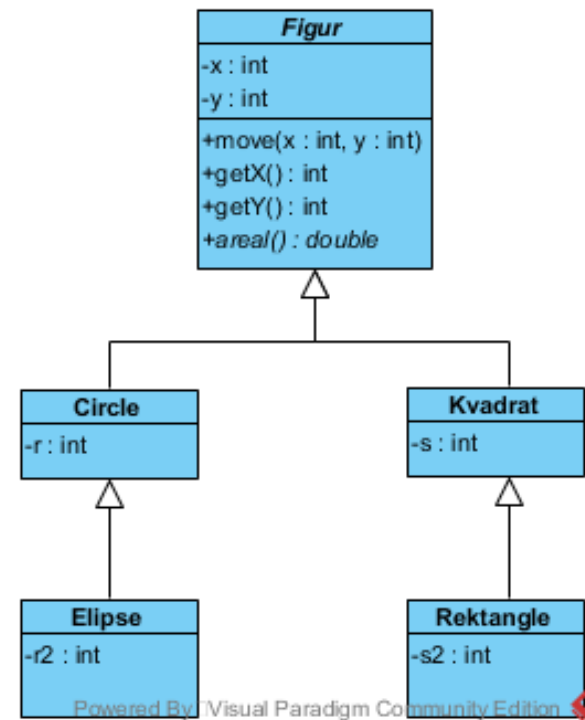
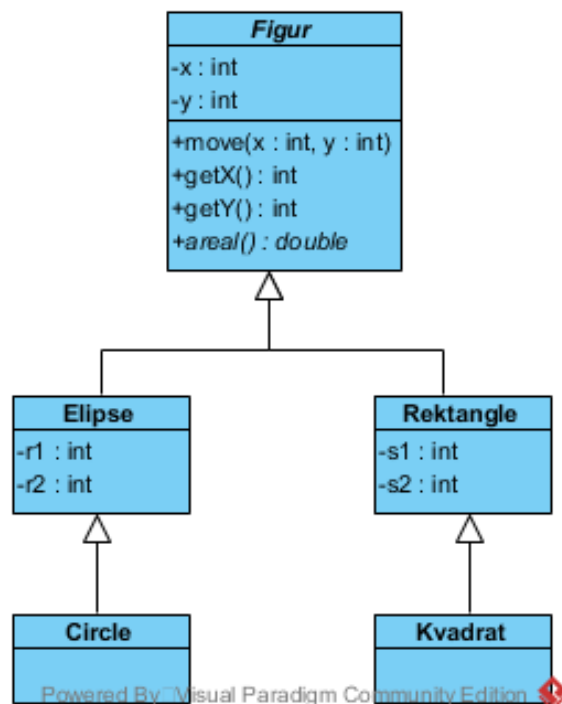
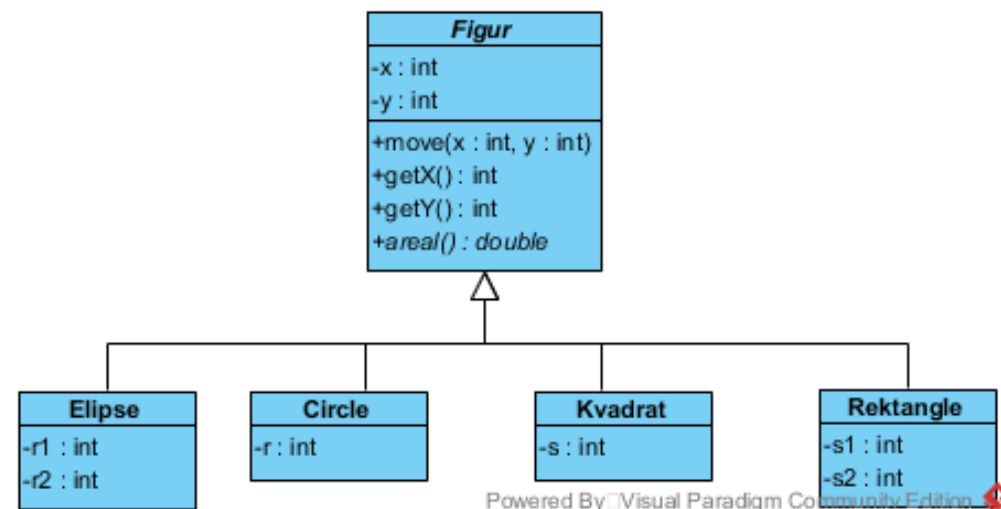
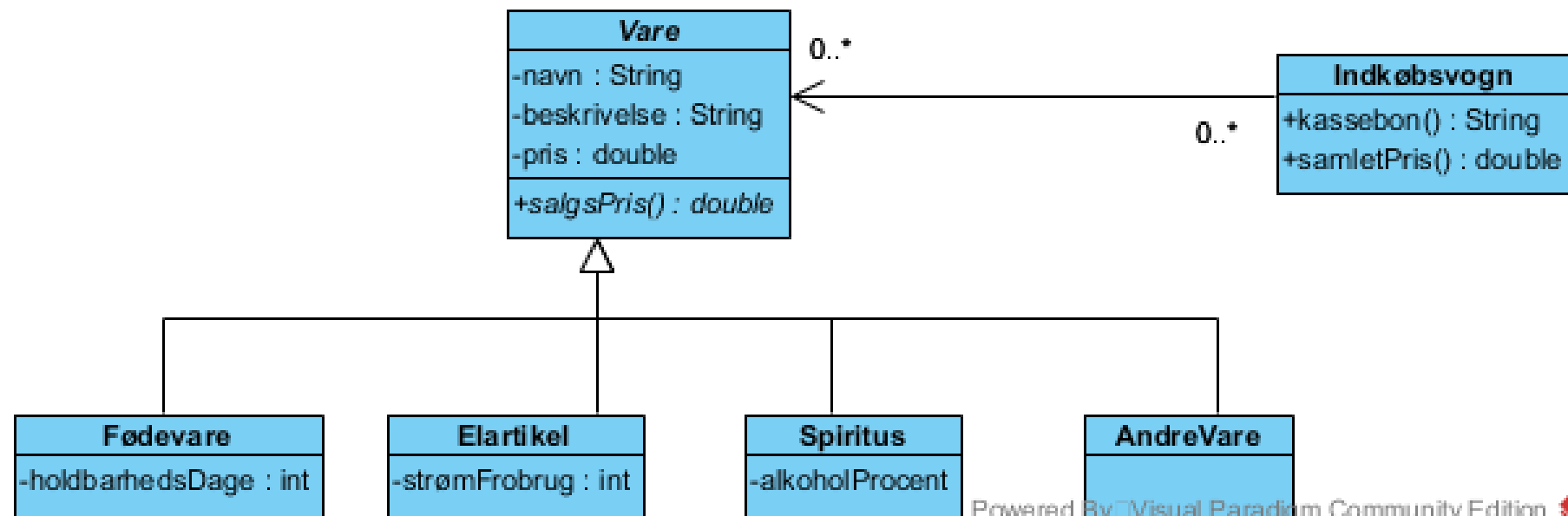


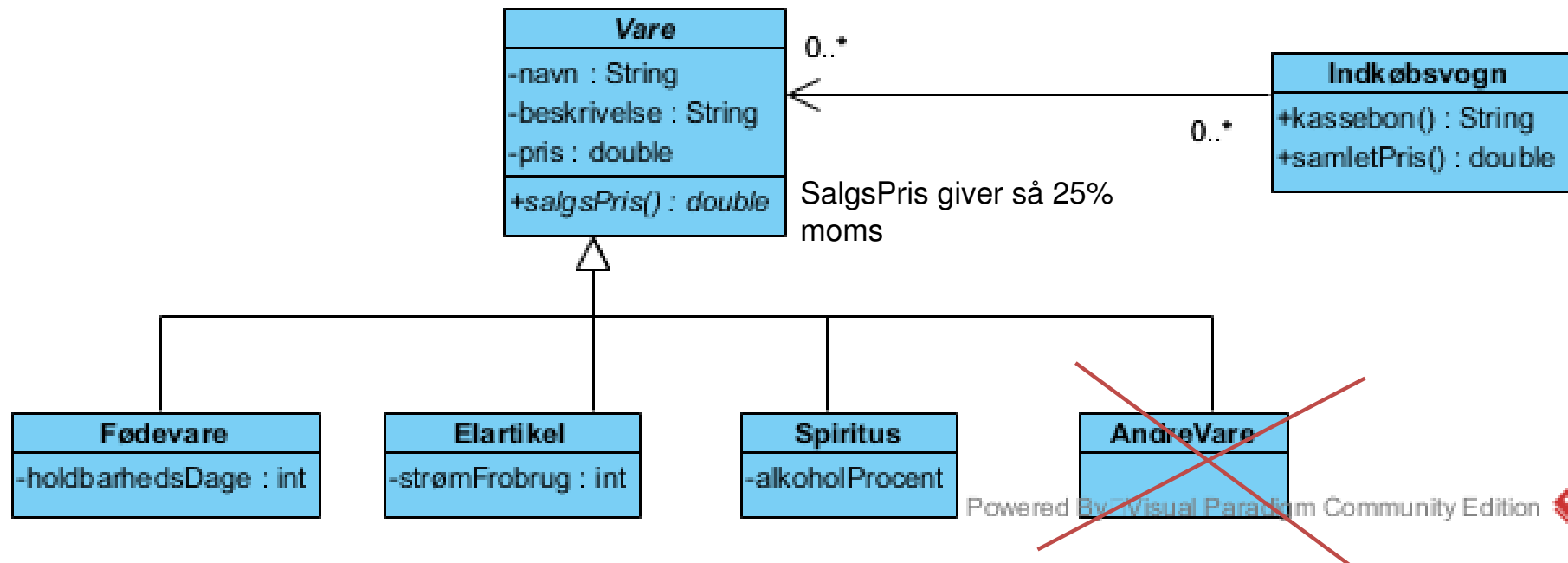
Interfaces

eksemplet/orden med Comparable





Vare er ikke abstract



Gennemsnit (banksaldi)

```
/** Computes average of balance. <br/>  
 * Requires: objects.length > 0.  
 */  
public static double average(BankAccount[] objects)  
{  
    double sum = 0;  
    for (BankAccount obj : objects) {  
        sum = sum + obj.getBalance();  
    }  
    return sum / objects.length;  
}
```

Gennemsnit (landearealer)

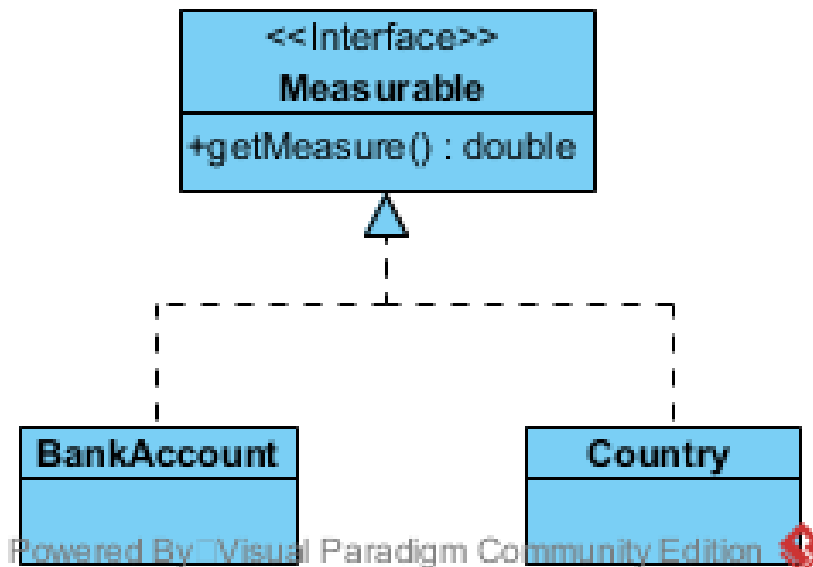
```
/** Computes average of area. <br/>  
 * Requires: objects.length > 0.  
 */  
public static double average(Country[] objects)  
{  
    double sum = 0;  
    for (Country obj : objects) {  
        sum = sum + obj.getArea();  
    }  
    return sum / objects.length;  
}
```

Erkæring af Interface Type

```
public interface Measurable {  
    double getMeasure();  
}
```

I et Interface er alle
metoder default public

Interface Measurable



BankAccount og Country skal så begge implementere metoden getMeasure

Polymorfisme (på metoder)

```
Measurable meas;
```

```
meas = new BankAccount(1000);
```

```
// ...eller...
```

```
meas = new Country("Denmark",  
                   43094);
```

```
double m = meas.getMeasure();
```

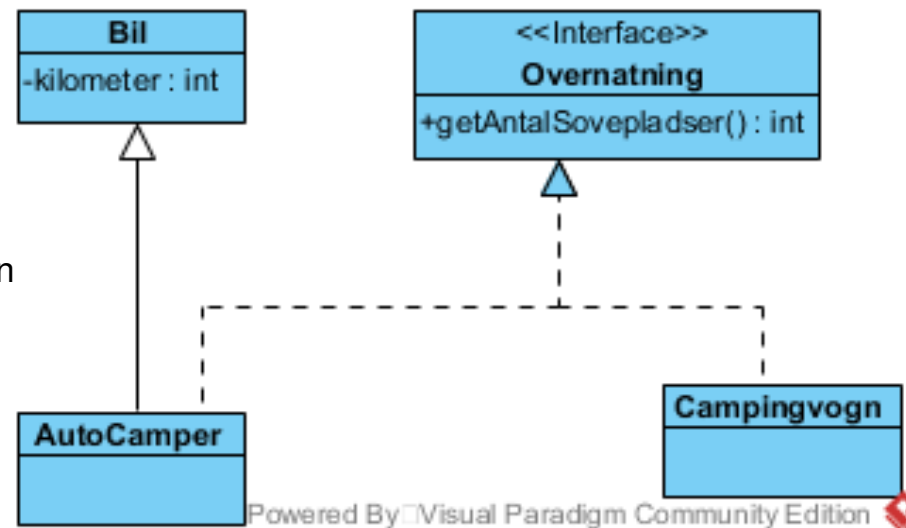
Interface

- Et interface er en type, der ikke kan laves objekter af
- Et interface er en type, der ikke indeholder feltvariable
- Et interface indeholder en **fælles "grænseflade"** og kan (*bør*) **ikke indeholde implementation**
- En type skal være et interface, når
 - Den modellerer en abstraktion uden implementation
- Et interface angives med nøgleordet **interface**

```
public interface Navn{...}
```
- Alle metoder i interface er **public**
- Et interface har ikke en constructor

Eksempel på interface

Ej muligt at lave multipel nedarvning i Java, MEN man kan implementere mange interfaces



```

public static void main(String[] args) {
    ArrayList<Overnatning> overnatninger = new ArrayList<Overnatning>();
    overnatninger.add(new Campingvogn(6));
    overnatninger.add(new AutoCamper(12300,4));
    int antal = 0;
    for (Overnatning on: overnatninger) {
        antal += on.getAntalSovepladser();
    }
    System.out.println(antal);
}

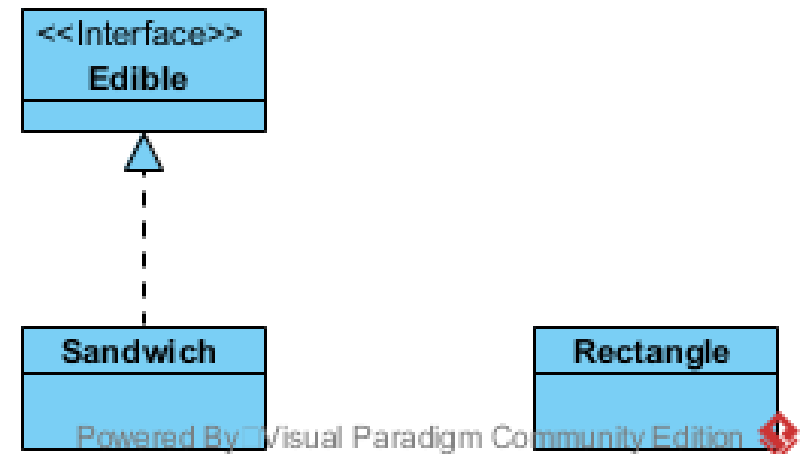
```

Quiz

```
Sandwich sub = new Sandwich(); // implementerer Edible
Rectangle sandwichBox = new Rectangle(10, 10, 25);
Edible e = null;
```

Hvilke statements er gyldige?

- a) e = sub;
- b) sub = e;
- c) sub = (Sandwich) e;
- d) sub = (Sandwich) sandwichBox;
- e) e = cerealBox;
- f) e = (Rectangle) sandwichBox;



Comparable

```
public interface Comparable<T> {  
    /**  
     * returns a negative integer, zero, or a  
     * positive integer as this object is less  
     * than, equal to, or greater than the  
     * specified object.  
     */  
    public int compareTo(T obj);  
}
```

Sammenligning String

`s1.compareTo(s2) < 0` når `s1` kommer før `s2`

`s1.compareTo(s2) = 0` når `s1` og `s2` er ens

`s1.compareTo(s2) > 0` når `s1` kommer efter `s2`

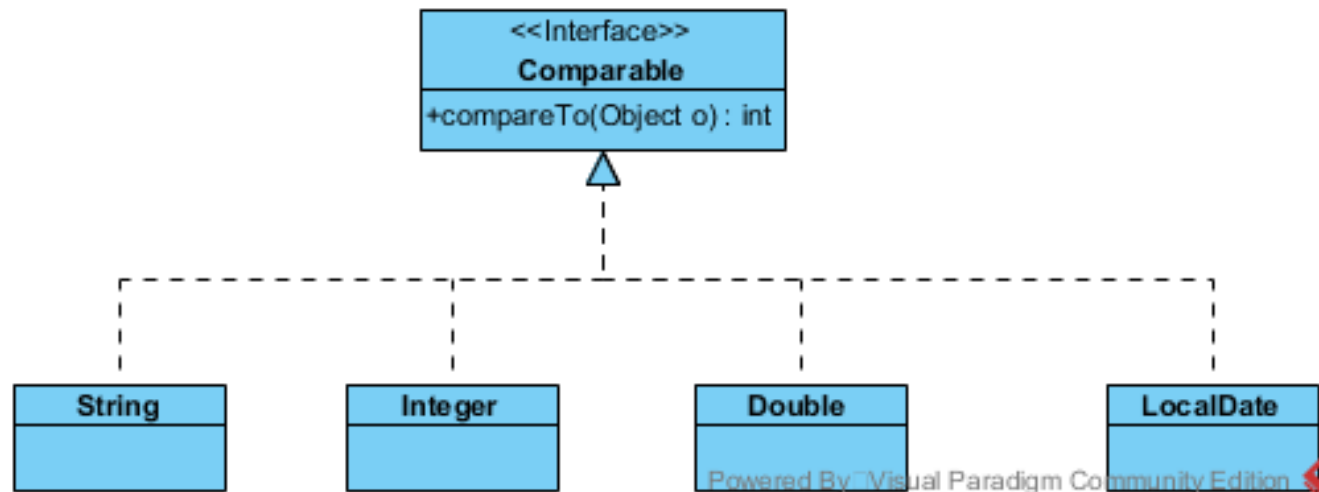
Eksempel:

`"Hanne".compareTo("Hans")` er `< 0`

`"Hans".compareTo("Hanne")` er `> 0`

`"Hans".compareTo("Hans")` er `0`

Comparable



En lang række klasser
implementerer allerede
Comparable interfacet

Comparable

Lad dine egne klasser implementere Comparable interfacet.

```
public class Person implements Comparable<Person> {  
    private String name;  
  
    ...  
    @Override  
    public int compareTo(Person p) {  
        int comp = this.name.compareTo(p.getName());  
        return comp;  
    }  
}
```

Her er personer ordnet efter deres navne